

Diseña un plan de implementación que cubra los aspectos de herramientas requeridas, diseño de UI/UX e implementación (dame la estructura de carpetas y archivos que se tendrán que agregar o modificar, más no el código) para trabajar con un proyecto en Flutter con Dart. El plan es crear una aplicación multiplataforma (Android, iOS, Window), utilizando Antigravity de Google como editor de código, que tenga conexión a una base de datos en Firebase Console. Usar estándar **no utilizar la opción de producción** en a, no utilizar analíticas. A continuación las especificaciones.

ESPECIFICACIONES DEL PROYECTO EN FIREBASE CONSOLE:

Nombre del proyecto	BD-CRUD-Freddys0442
ID del proyecto	bd-crud-freddys0442
Número del proyecto	211200640128

Se planea que la aplicación actúe como un servicio de comida a domicilio, similar a DiDi Foods, Uber Eats o Rappi, en donde el usuario puede realizar una orden en línea y su pedido es comunicado a quienes lo preparan y después a los repartidores para que este llegue a manos del usuario.

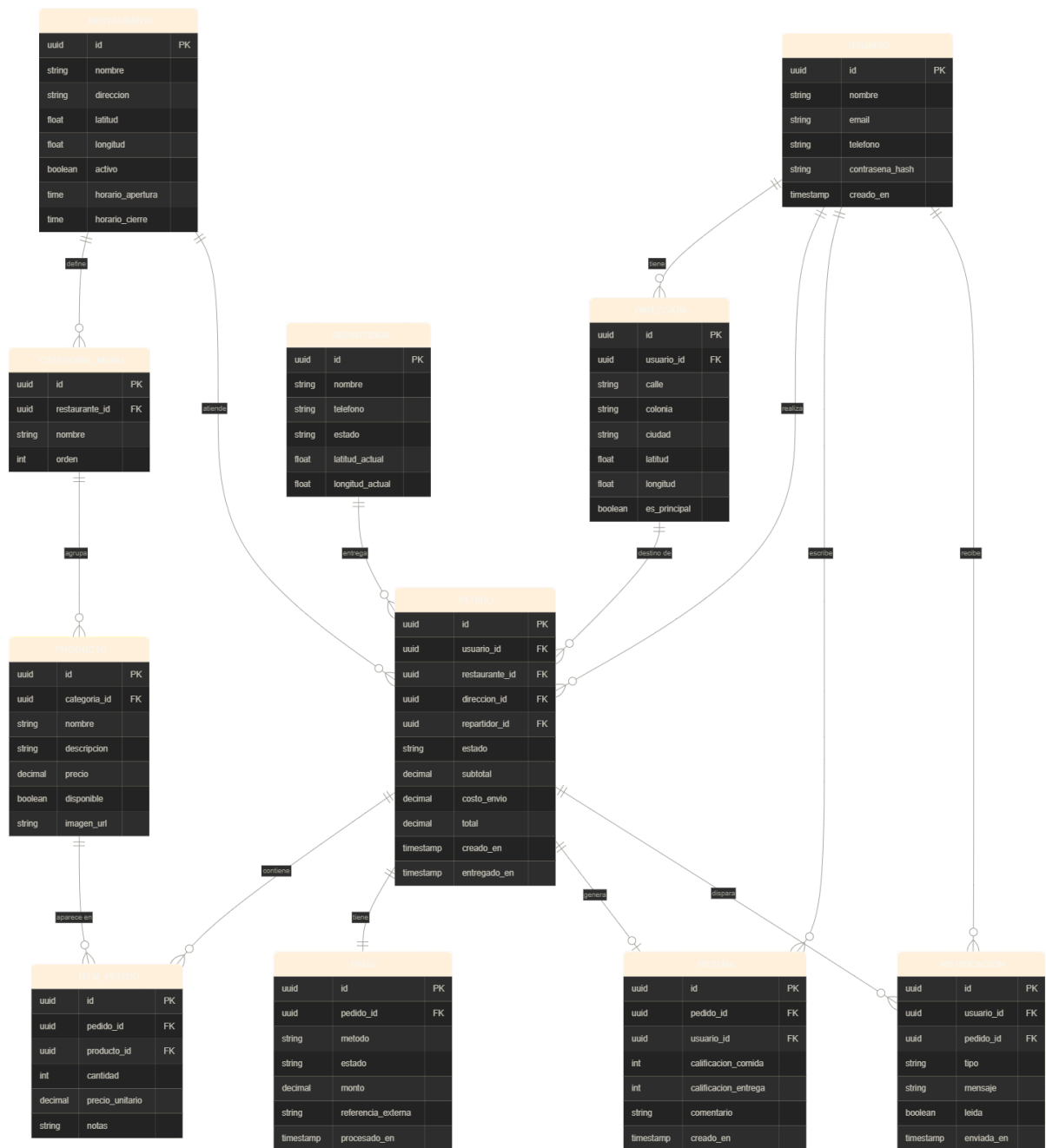
IDENTIDAD VISUAL DEL PROYECTO:

- ★ Apariencia General: Tema oscuro, color de fondo hex #1e1e1e.
- ★ Acentos principales basados en la paleta de color de los distintos personajes del Mega Pizzaplex.
- ★ Ambientación/Inspiración: FNaF Security Breach y sus personajes.
- ★ Jugar con la disposición de los elementos para crear algo juguetón (no demasiado, mantener sobriedad de ser necesario) y atractivo.

Entonces, esta aplicación debe contar con el flujo necesario para que este proceso se complete, comenzando por el registro/inicio de sesión del usuario tras mostrar una pequeña animación al abrir la aplicación. Deben almacenarse y validar los datos para garantizar la seguridad del usuario. Además, el sistema debe ser capaz de generar una ramificación a la hora de iniciar sesión para mostrar lo adecuado según si el usuario que ha iniciado sesión puede tener control sobre el sistema (un administrador que tiene los permisos para realizar acciones CRUD. Estos administradores se designarán mediante la función designada como autenticación en Firebase Console) o un usuario normal (cliente) de forma que no muestre, por ejemplo, el panel administrativo para gestionar el menú a una persona que solo quiere realizar un pedido. Así también, se debe mantener la sesión iniciada a menos que el usuario indique lo contrario y cierre la sesión (aunque si esto ocurre, es necesario que se guarden los datos y cambios realizados por este).

Una vez que se haya completado este primer paso, dependiendo de la ruta que se tome (usuario común/administrador), se mostrará la página de inicio o el panel de control por defecto.

ESTRUCTURA RECOMENDADA PARA LA BASE DE DATOS:



El esquema cubre **11 tablas** que simulan el ciclo completo. Aquí un resumen de los grupos lógicos:

Gestión de usuarios y direcciones — **USUARIO** y **DIRECCION** permiten que un cliente tenga múltiples domicilios guardados, con coordenadas para el cálculo de rutas y costos de envío.

Catálogo del restaurante — **RESTAURANTE**, **CATEGORIA_MENU** y **PRODUCTO** modelan la estructura jerárquica del menú (ej: *Pizzas* → *Margarita*), con campos de disponibilidad y horario de atención.

Ciclo del pedido — **PEDIDO** es la entidad central. Relaciona al usuario, restaurante, dirección de entrega y repartidor asignado. El campo **estado** recorre el ciclo: **pendiente** → **confirmado** → **en_preparacion** → **en_camino** → **entregado**. **ITEM_PEDIDO** guarda la foto fija del precio al momento de comprar (importante: no se debe tomar el precio actual del producto, sino el que tenía al pedirlo).

Pagos — **PAGO** tiene relación uno a uno con **PEDIDO**. El campo **referencia_externa** almacena el ID que devuelve la pasarela de pago (Stripe, Conekta, etc.) para conciliación.

Repartidor — **REPARTIDOR** guarda su ubicación en tiempo real (**latitud_actual**, **longitud_actual**) para el tracking del pedido.

Post-entrega — **RESENA** separa la calificación de la comida y la de la entrega, lo cual permite retroalimentación diferenciada para el restaurante y el repartidor. **NOTIFICACION** permite registrar todos los eventos enviados al usuario (push, SMS, email) para auditoría y reenvío.

PK Llave primaria FK Llave foránea IDX Indexado

usuario8 cols

id	UUID	PK
nombre	VARCHAR(120)	
email	VARCHAR(255)	IDX
telefono	VARCHAR(20)	
contrasena_hash	TEXT	
activo	BOOLEAN	
creado_en	TIMESTAMPZ	
actualizado_en	TIMESTAMPZ	

direccion10 cols

id	UUID	PK
usuario_id	UUID	FK IDX
alias	VARCHAR(60)	
calle	VARCHAR(200)	
colonia	VARCHAR(120)	
ciudad	VARCHAR(100)	
codigo_postal	CHAR(5)	
latitud	DECIMAL(9,6)	
longitud	DECIMAL(9,6)	
es_principal	BOOLEAN	

restaurante13 cols

id	UUID	PK
nombre	VARCHAR(150)	
descripcion	TEXT	
direccion	TEXT	
latitud	DECIMAL(9,6)	
longitud	DECIMAL(9,6)	
telefono	VARCHAR(20)	
logo_url	TEXT	
activo	BOOLEAN	
horario_apertura	TIME	
horario_cierre	TIME	
tiempo_entrega_min	SMALLINT	
radio_entrega_km	DECIMAL(5,2)	

categoria_menu7 cols

id	UUID	PK
restaurante_id	UUID	FK IDX
nombre	VARCHAR(80)	
descripcion	VARCHAR(255)	
imagen_url	TEXT	
orden	SMALLINT	
activa	BOOLEAN	

producto10 cols

id	UUID	PK
categoria_id	UUID	FK IDX
nombre	VARCHAR(120)	
descripcion	TEXT	
precio	DECIMAL(10,2)	
imagen_url	TEXT	
disponible	BOOLEAN	
calorias	SMALLINT	
tiempo_prep_min	SMALLINT	
es_vegano	BOOLEAN	

pedido13 cols

id	UUID	
usuario_id	UUID	
restaurante_id	UUID	
direccion_id	UUID	
repartidor_id	UUID	
estado	ENUM('pendiente','confirmado','en_preparacion','en_camino','entregado','cancelado')	
subtotal	DECIMAL(10,2)	
costo_envio	DECIMAL(10,2)	
descuento	DECIMAL(10,2)	
total	DECIMAL(10,2)	
notas	TEXT	
creado_en	TIMESTAMPZ	
entregado_en	TIMESTAMPZ	

item_pedido8 cols

id	UUID	PK
pedido_id	UUID	FK IDX
producto_id	UUID	FK
nombre_snapshot	VARCHAR(120)	
precio_unitario	DECIMAL(10,2)	
cantidad	SMALLINT	
subtotal	DECIMAL(10,2)	
notas	TEXT	

repartidor8 cols

id	UUID	
nombre	VARCHAR(120)	
telefono	VARCHAR(20)	
estado	ENUM('disponible','en_entrega','inactivo')	
latitud_actual	DECIMAL(9,6)	
longitud_actual	DECIMAL(9,6)	
ubicacion_actualizada	TIMESTAMPZ	
calificacion_promedio	DECIMAL(3,2)	

pago8 cols

id	UUID	
pedido_id	UUID	
metodo	ENUM('tarjeta','efectivo','transferencia','wallet')	
estado	ENUM('pendiente','aprobado','rechazado','reembolsado')	
monto	DECIMAL(10,2)	
moneda	CHAR(3)	
referencia_externa	VARCHAR(200)	
procesado_en	TIMESTAMPZ	

resena7 cols

id	UUID	PK
pedido_id	UUID	FK IDX
usuario_id	UUID	FK
cal_comida	SMALLINT	
cal_entrega	SMALLINT	
comentario	TEXT	
creado_en	TIMESTAMPZ	

notificacion8 cols

id	UUID	PK
usuario_id	UUID	FK IDX
pedido_id	UUID	FK
tipo	ENUM('push','sms','email')	
canal	VARCHAR(60)	
mensaje	TEXT	
leida	BOOLEAN	
enviada_en	TIMESTAMPZ	IDX

SCRIPT SQL:

```
-- =====
-- MegaPizzaplex — Schema SQL
-- Base de datos: PostgreSQL 15+
-- Generado: 2026-05-11
-- =====

-- Extensión para UUID
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-----
-- TIPOS ENUM
-----

CREATE TYPE estado_pedido AS ENUM (
    'pendiente',
    'confirmado',
    'en_preparacion',
    'en_camino',
    'entregado',
    'cancelado'
);

CREATE TYPE estado_pago AS ENUM (
    'pendiente',
    'aprobado',
    'rechazado',
    'reembolsado'
);

CREATE TYPE metodo_pago AS ENUM (
    'tarjeta',
    'efectivo',
    'transferencia',
    'wallet'
);

CREATE TYPE estado_repartidor AS ENUM (
    'disponible',
    'en_entrega',
    'inactivo'
);

CREATE TYPE tipo_notificacion AS ENUM (
    'push',
    'sms',
    'email'
);

-----
-- GRUPO: USUARIOS
-----

CREATE TABLE usuario (
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    nombre      VARCHAR(120)  NOT NULL,
    email       VARCHAR(255)  NOT NULL,
    telefono    VARCHAR(20),
    contraseña_hash TEXT      NOT NULL,
    activo      BOOLEAN       NOT NULL DEFAULT TRUE,
```

```

    creado_en    TIMESTAMPTZ    NOT NULL DEFAULT NOW(),
    actualizado_en TIMESTAMPTZ    NOT NULL DEFAULT NOW()
);

```

```

CREATE UNIQUE INDEX idx_usuario_email ON usuario (email);

```

```

CREATE TABLE direccion (
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    usuario_id  UUID          NOT NULL REFERENCES usuario (id) ON DELETE CASCADE,
    alias       VARCHAR(60),
    calle       VARCHAR(200)  NOT NULL,
    colonia     VARCHAR(120)  NOT NULL,
    ciudad      VARCHAR(100)  NOT NULL,
    codigo_postal CHAR(5)     NOT NULL,
    latitud     DECIMAL(9, 6) NOT NULL,
    longitud    DECIMAL(9, 6) NOT NULL,
    es_principal BOOLEAN      NOT NULL DEFAULT FALSE
);

```

```

CREATE INDEX idx_direccion_usuario ON direccion (usuario_id);

```

```

-- GRUPO: CATÁLOGO

```

```

CREATE TABLE restaurante (
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    nombre      VARCHAR(150)  NOT NULL,
    descripcion  TEXT,
    direccion   TEXT          NOT NULL,
    latitud     DECIMAL(9, 6) NOT NULL,
    longitud    DECIMAL(9, 6) NOT NULL,
    telefono    VARCHAR(20),
    logo_url    TEXT,
    activo      BOOLEAN       NOT NULL DEFAULT TRUE,
    horario_apertura TIME     NOT NULL DEFAULT '08:00',
    horario_cierre TIME       NOT NULL DEFAULT '22:00',
    tiempo_entrega_min SMALLINT NOT NULL DEFAULT 30,
    radio_entrega_km DECIMAL(5, 2) NOT NULL DEFAULT 5.00
);

```

```

CREATE TABLE categoria_menu (
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    restaurante_id UUID       NOT NULL REFERENCES restaurante (id) ON DELETE CASCADE,
    nombre      VARCHAR(80)   NOT NULL,
    descripcion  VARCHAR(255),
    imagen_url   TEXT,
    orden       SMALLINT      NOT NULL DEFAULT 0,
    activa      BOOLEAN       NOT NULL DEFAULT TRUE
);

```

```

CREATE INDEX idx_categoria_restaurante ON categoria_menu (restaurante_id);

```

```

CREATE TABLE producto (
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    categoria_id UUID       NOT NULL REFERENCES categoria_menu (id) ON DELETE CASCADE,
    nombre      VARCHAR(120)  NOT NULL,
    descripcion  TEXT,
    precio      DECIMAL(10, 2) NOT NULL CHECK (precio >= 0),
    imagen_url   TEXT,

```

```

    disponible    BOOLEAN    NOT NULL DEFAULT TRUE,
    calorias      SMALLINT,
    tiempo_prep_min SMALLINT,
    es_vegano     BOOLEAN    NOT NULL DEFAULT FALSE
);

```

```

CREATE INDEX idx_producto_categoria ON producto (categoria_id);

```

```

-- -----
-- GRUPO: OPERACIÓN
-- -----

```

```

CREATE TABLE repartidor (
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    nombre      VARCHAR(120)  NOT NULL,
    telefono    VARCHAR(20)   NOT NULL,
    estado      estado_repartidor NOT NULL DEFAULT 'inactivo',
    latitud_actual DECIMAL(9, 6),
    longitud_actual DECIMAL(9, 6),
    ubicacion_actualizada TIMESTAMPTZ,
    calificacion_promedio DECIMAL(3, 2) CHECK (calificacion_promedio BETWEEN 0 AND 5)
);

```

```

-- -----
-- GRUPO: PEDIDOS
-- -----

```

```

CREATE TABLE pedido (
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    usuario_id  UUID          NOT NULL REFERENCES usuario (id),
    restaurante_id UUID      NOT NULL REFERENCES restaurante (id),
    direccion_id UUID        NOT NULL REFERENCES direccion (id),
    repartidor_id UUID        REFERENCES repartidor (id),
    estado      estado_pedido NOT NULL DEFAULT 'pendiente',
    subtotal    DECIMAL(10, 2) NOT NULL CHECK (subtotal >= 0),
    costo_envio DECIMAL(10, 2) NOT NULL DEFAULT 0 CHECK (costo_envio >= 0),
    descuento   DECIMAL(10, 2) NOT NULL DEFAULT 0 CHECK (descuento >= 0),
    total       DECIMAL(10, 2) NOT NULL CHECK (total >= 0),
    notas       TEXT,
    creado_en   TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
    entregado_en TIMESTAMPTZ
);

```

```

CREATE INDEX idx_pedido_usuario ON pedido (usuario_id);
CREATE INDEX idx_pedido_restaurante ON pedido (restaurante_id);
CREATE INDEX idx_pedido_estado ON pedido (estado);
CREATE INDEX idx_pedido_creado ON pedido (creado_en DESC);

```

```

-- -----
CREATE TABLE item_pedido (
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    pedido_id   UUID          NOT NULL REFERENCES pedido (id) ON DELETE CASCADE,
    producto_id UUID          REFERENCES producto (id) ON DELETE SET NULL,
    nombre_snapshot VARCHAR(120) NOT NULL,
    precio_unitario DECIMAL(10, 2) NOT NULL CHECK (precio_unitario >= 0),
    cantidad    SMALLINT      NOT NULL CHECK (cantidad > 0),
    subtotal    DECIMAL(10, 2) NOT NULL CHECK (subtotal >= 0),
    notas       TEXT
);

```

```

CREATE INDEX idx_item_pedido ON item_pedido (pedido_id);

```

```

-- -----
CREATE TABLE pago (

```

```

id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
pedido_id   UUID          NOT NULL UNIQUE REFERENCES pedido (id) ON DELETE CASCADE,
metodo      metodo_pago   NOT NULL,
estado      estado_pago   NOT NULL DEFAULT 'pendiente',
monto       DECIMAL(10, 2) NOT NULL CHECK (monto > 0),
moneda      CHAR(3)       NOT NULL DEFAULT 'MXN',
referencia_externa VARCHAR(200),
procesado_en TIMESTAMPTZ
);

```

```

CREATE INDEX idx_pago_pedido ON pago (pedido_id);

```

```

-- -----

```

```

CREATE TABLE resena (
id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
pedido_id   UUID          NOT NULL UNIQUE REFERENCES pedido (id) ON DELETE CASCADE,
usuario_id  UUID          NOT NULL REFERENCES usuario (id),
cal_comida  SMALLINT      NOT NULL CHECK (cal_comida BETWEEN 1 AND 5),
cal_entrega SMALLINT      NOT NULL CHECK (cal_entrega BETWEEN 1 AND 5),
comentario  TEXT,
creado_en   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE INDEX idx_resena_pedido ON resena (pedido_id);

```

```

-- -----

```

```

CREATE TABLE notificacion (
id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
usuario_id  UUID          NOT NULL REFERENCES usuario (id) ON DELETE CASCADE,
pedido_id   UUID          REFERENCES pedido (id) ON DELETE SET NULL,
tipo        tipo_notificacion NOT NULL,
canal        VARCHAR(60),
mensaje     TEXT          NOT NULL,
leida        BOOLEAN       NOT NULL DEFAULT FALSE,
enviada_en   TIMESTAMPTZ   NOT NULL DEFAULT NOW()
);

```

```

CREATE INDEX idx_notificacion_usuario ON notificacion (usuario_id);
CREATE INDEX idx_notificacion_enviada ON notificacion (enviada_en DESC);

```

```

-- -----
-- FUNCIÓN: actualizar timestamp automáticamente
-- -----

```

```

CREATE OR REPLACE FUNCTION set_actualizado_en()
RETURNS TRIGGER AS $$
BEGIN
    NEW.actualizado_en = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_usuario_actualizado
BEFORE UPDATE ON usuario
FOR EACH ROW EXECUTE FUNCTION set_actualizado_en();

```

```

-- =====
-- Fin del schema — MegaPizzaplex
-- =====

```